CHAPTER 2

LEARNING LOCAL HIGHER-ORDER INTERACTIONS

2.1 Introduction

Higher-order interactions (HOIs) occur when three or more variables interact simultaneously such that the combined effect cannot be explained by summing or merely aggregating pairwise interactions. These interactions capture the synergistic—or sometimes antagonistic—effects that arise only when multiple components work together, leading to emergent behavior that cannot be understood by studying simpler two-variable relationships alone. HOIs are a pervasive aspect of complex systems and are evident in numerous domains—such as neural activity [17, 18], ecological systems [19, 20], and even social collaborations [21]—variables interact in ways that go beyond simple pairwise relationships. In complex systems, variables often interact together in complicated ways [22, 23]. Traditional methods for exploring relationships, like Pearson's or Spearman's correlation, mutual information, or total correlation, have been useful for examining pairwise dependencies but do not scale well when considering all possible HOIs, which grow as $O(2^p)$ for p variables.

Just as HOIs reveal that the whole can be greater than the sum of its parts in neural, ecological, and social systems, a similar level of intricate interplay characterizes modern artificial intelligence. In the realm of machine learning, especially with neural networks, uncovering such complex interactions is pivotal for improving explainability and interpretability. Explainability and interpretability are fundamental for trust and adoption of machine learning algorithms, yet interpreting even relatively simple neural networks on complex data is challenging. A recent survey [24] categorizes inner neural network interpretability techniques based on which part of the network's computational graph they explain—weights, neurons, subnetworks, or latent representations. Each of these groups can be categorized further as intrinsic and post hoc, where the former involves training models in a manner

to be more easily interpretable, and the latter aims to interpret the model after training is completed. Additional work focused on discovering novel behaviors in neural networks include identifying where knowledge is stored in LLM weights [25, 26], labeling nodes with natural language descriptions [27], and editing classifiers to interpret a concept in a new manner given an example [28]. All of these methods require either an input of interest or the class labels and are thus supervised.

Despite these advances, HOIs remain hard to uncover. The primary challenges include:

- **Computational complexity:** As the number of variables increases, the number of possible interactions grows exponentially.
- **Assumption of linearity:** Many traditional tools assume that linear correlations are sufficient. However, linear approximations tend to work only for data points near the average and often fail when deviations become large. This poses a problem in fields like precision medicine, where we hope to have predictive power not just for the average user, but for all users.
- **Inconsistent HOIs:** Assuming that HOIs occur uniformly across an entire dataset can obscure class-specific or context-dependent interactions. This overgeneralization introduces additional noise, making it harder to detect and interpret the unique interaction patterns that only appear under certain conditions.

One strategy to overcome the computational complexity is dimensionality reduction, which transforms high-dimensional data into a simpler representation while hopefully retaining key relationships from the data. Within this framework, methods such as Cor-relation Ex-planation (CorEx) have been developed to construct informative representations of variable relationships in high-dimensional data.

The original CorEx framework has evolved into variants such as Linear CorEx and Bio CorEx. Linear CorEx estimates multivariate Gaussian distributions by identifying independent latent factors that explain correlations among observed variables [29]. It uses a

modular inductive prior that favors block-diagonal covariance structures, effectively identifying clusters governed by a few latent factors. Bio CorEx, on the other hand, is designed to handle challenges inherent in biomedical data, such as missing data, continuous variables, and severe undersampling [30]. However, both variants assume that the variable interactions are uniform across the entire data manifold, and Linear CorEx is a linear method.

Here we present Local CorEx, which addresses both of these limitations by partitioning the data prior to estimating variable interactions. Breaking the data up in this manner transforms a linear method into a piecewise linear method, which is far superior at modeling non linear phenomena and can improve the accuracy by increasing the number of model fits. Additionally, this approach enables the detection of locally varying HOIs and allows the method to be applied in an unsupervised manner, when label information is not available, although it can incorporate labels when provided.

Part of this work was previously published in [16]. Our specific contributions are as follows:

- We derive a novel method for estimating local variable interactions called Local CorEx (Section 2.2).
- We demonstrate on synthetic data that Local CorEx is robust to hyperparameter selection and outperforms previous methods when HOIs vary across the data manifold (Section 2.3.1).
- We show the utility of Local CorEx in extracting plausible HOIs from real-world data (Sections 2.3.2 & 2.3.3).
- We interpret the inner workings of neural network classifiers using Local CorEx, uncovering sets of hidden nodes that are critical for accurately classifying a local cluster while leaving other clusters largely unaffected (Section 2.3.4 & 2.3.5).

2.2 Local CorEx

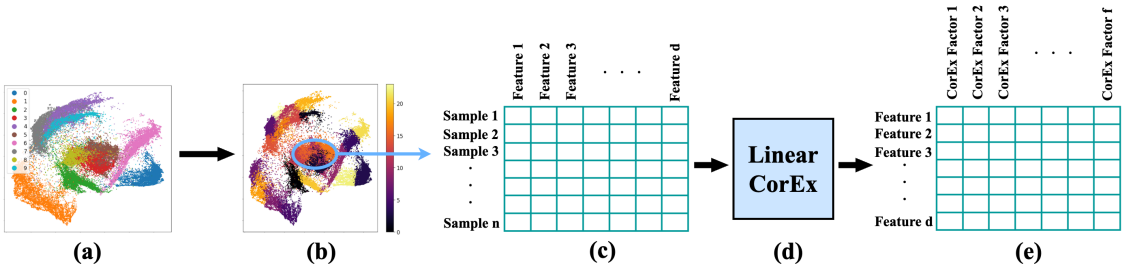


Fig. 2.1: Overview of the Local CorEx algorithm. *(a)* PHATE visualization of the MNIST dataset colored by class (i.e. digit) label. *(b)* k-means clustering is applied to the PHATE embedding to generate the local clusters. *(c-d)* A cluster is chosen and Linear CorEx is applied to learn the latent factor representation. *(e)* We visualize the mutual information between the learned CorEx latent factors and the original features to identify HOIs within this cluster.

Figure 2.1 gives an overview of Local CorEx. Local CorEx first uses PHATE [31] to create a low-dimensional embedding of the data that preserves the local and global structure of the data manifold(s). PHATE is a dimensionality reduction technique that is specialized for data visualization [31]. It learns the local structure via a specialized kernel function that learns pairwise similarities between data points. The pairwise kernel matrix is row-normalized to create a Markov transition matrix, called the diffusion operator [32]. The global structure of the data is then learned via diffusion by simulating t-step random walks between data points according to the normalized affinities. Finally, information distances are calculated between the diffused probabilities, and multidimensional scaling (MDS) [33] is used to preserve these distances in low dimensions.

To partition the data, we apply k-means clustering [34] to the PHATE embedding where k is chosen to be sufficiently large to ensure heterogeneity among the partitions according to a manual inspection of a 2D PHATE plot. This clustering method is akin to spectral clustering [35], which essentially applies k-means to the Laplacian Eigenmaps embedding. We choose this clustering approach over other methods because of PHATE's impressive visualization capabilities. After performing the clustering we can easily visualize the data using PHATE and color by clusters to see how the clusters relate to each other and if the partition makes visual sense.

After the partition has been created, we next apply either Linear or Bio CorEx to

each cluster to obtain the local variable interactions. We now describe how Linear and Bio CorEx work.

2.2.1 Information Theory

Notation

Let $X \equiv X_{1:p} \equiv \{X_1, X_2, \cdots, X_p\}$ denote a vector of p observed variables and let $Z \equiv Z_{1:m} \equiv \{Z_1, Z_2, \cdots, Z_m\}$ denote a vector of m latent variables. Instances of X and Z are denoted in lowercase with $x = (x_1, x_2, \cdots, x_p)$ and $z = (z_1, z_2, \cdots, z_m)$, respectively.

Formal Definitions

We consider several information-theoretic measures including:

- **Differential Entropy**: For an absolutely continuous random vector X with density $p(x)$, the *differential entropy* is

$$H(X) \;=\; -\int_{\mathbb{R}^p} p(x)\, \log p(x)\, \mathrm{d}x.$$

  This extends Shannon's discrete entropy to continuous distributions but can take negative values when densities can exceed 1 in regions of small measure.

- **Mutual Information**: For two random vectors X and Z with joint density $p(x, z)$ and marginals $p(x)$, $p(z)$, the *mutual information* is

$$I(X; Z) \;=\; \int p(x, z)\, \log \frac{p(x, z)}{p(x)\, p(z)}\, \mathrm{d}x\, \mathrm{d}z \;=\; H(X) + H(Z) - H(X, Z).$$

  Equivalently, $I(X; Z) = D_{KL}(p(x, z) \,\|\, p(x)p(z))$. It is nonnegative and equals zero if and only if X and Z are independent.

- **Total Correlation**: Also known as *multivariate mutual information* or *multi-information*, the *total correlation* among $X_{1:p}$ is

$$TC(X) \;=\; \sum_{i=1}^{p} H(X_i) \;-\; H(X) \;=\; D_{KL}\big(p(x) \,\big\|\, \prod_{i=1}^{p} p(x_i)\big).$$

  It measures the total amount of statistical dependency or redundancy among the p variables.

- **Conditional Entropy**: The entropy of X conditioned on Z is

$$H(X \mid Z) \;=\; \int p(z)\, H\big(X \mid Z{=}z\big)\, \mathrm{d}z, \quad H(X \mid Z{=}z) = -\int p(x \mid z)\, \log p(x \mid z)\, \mathrm{d}x.$$

- **Conditional Total Correlation**: The total correlation of X given Z is

$$TC(X \mid Z) \;=\; \int p(z) \Big[\sum_{i=1}^{p} H(X_i \mid Z{=}z) \;-\; H(X \mid Z{=}z) \Big]\, \mathrm{d}z.$$

Intuitive Interpretations

- **Differential Entropy as Spread**: Differential entropy quantifies the "spread" of a continuous distribution. For example, under the constraint that a density is supported on an interval $[a, b]$, the *uniform distribution* $p(x) = 1/(b-a)$ maximizes $H(X)$ with value $\log(b-a)$. By contrast, a Dirac delta at a point a has $H(\delta(\cdot - a)) = -\infty$, reflecting zero dispersion.

- **Mutual Information as Non-linear Correlation**: Mutual information measures the reduction in uncertainty about X given knowledge of Z. Unlike Pearson's correlation, $I(X; Z)$ captures all statistical dependencies and is measured in "nats" or "bits" depending on log-base.

- **Total Correlation as Multivariate Dependence** Total correlation generalizes mutual information to p variables, aggregating pairwise and higher-order dependencies. A zero value indicates complete independence, while large values signal strong redundancy among the set.

2.2.2 Correlation Explanation

Correlation Explanation (CorEx) seeks a set of latent factors $Z = \{Z_1, \ldots, Z_m\}$;that explains as much of the multivariate dependency structure in observed data $X = \{X_1, \ldots, X_p\}$ as possible. This is achieved by optimizing an information-theoretic objective grounded in *total correlation*. Formally, CorEx defines the *explained total correlation*

$$TC(X; Z) \;=\; TC(X) \;-\; TC(X \mid Z),$$

and aims to maximize this quantity. Equivalently, one may write the CorEx objective in minimization form as

$$\min_{p(z|x)} \big[\, TC(X \mid Z) \;+\; TC(Z) \big],$$

where the global minimum of zero indicates that the Zs fully explain all dependencies in X. Under this view, **maximizing** $TC(X; Z)$ is equivalent to **minimizing** the residual $TC(X \mid Z)$, yielding latent factors that explain maximal redundancy. Moreover, the objective decomposes into sums of mutual information terms, offering insight into how each factor contributes. A key advantage of CorEx is its straightforward interpretability: each latent factor Z_j "explains" a quantifiable portion of total correlation in X, allowing one to rank factors by the magnitude of $TC(X; Z_j)$. This ranking facilitates feature selection and interpretive analysis.

Under this general framework, **Bio CorEx** [30] imposes no distributional form on X but restricts each Z_j to a discrete finite alphabet. A variational lower bound on $TC(X; Z)$ enables greedy, coordinate-ascent learning of one factor at a time—each iteration finds the next Z_j that maximizes explained TC conditioning on previously learned factors. This

makes it ideal to handle biomedical data challenges—such as missing entries, heterogeneous variable types, and severe under-sampling—while preserving interpretability.

In contrast, **Linear CorEx** [29] assumes X is jointly Gaussian and enforces $Z = WX + \epsilon$ with $\epsilon \sim \mathcal{N}(0, \Sigma)$. Under normality, TC depends only on covariances, so one can directly optimize

$$\min_{W,\Sigma} \; TC(X \mid Z) \; + \; TC(Z) \; + \; \sum_{i=1}^{p} Q_i,$$

where the Q_i are KL-based regularizers encouraging modularity of each Z_j with respect to individual X_i. This modularity improves the interpretability of the Linear CorEx factors with respect to the original feature space as each x_i is only affected by a single z_i. Linear CorEx thus identifies orthogonal, independent directions in data that explain maximal redundancy.

Both Linear and Bio CorEx are far faster then a brute force search for HOIs. Bio CorEx has a computational complexity of $O(m \times n \times N)$ where m is the number of latent factors, n is the number of variables, and N is the number of samples. Although Linear CorEx shares the same computational complexity, its factors can be learned in parallel.

Each factor's individual contribution is given by

$$TC(X; Z_j) \; = \; TC(X) \; - \; TC(X \mid Z_j, \{Z_{k \neq j}\}),$$

allowing factors to be ranked by the portion of redundancy they explain and serve as a measure of its importance. We can identify the predicted HOIs by examining the estimated mutual information between each latent factor and the original set of features. Here, the magnitude of the estimated mutual information can be used as a proxy for the strength or importance of the feature to the predicted HOI.

Local CorEx hyperparameters include:

1. The number of dimensions to include in the PHATE embedding of the data
2. The number of clusters for k-means
3. The number of CorEx latent factors to learn

4. For each CorEx factor, a threshold for when a feature is a part of a learned HOI.

To find the number of dimensions of the PHATE embedding for clustering, we have found that in most cases, unless there are few observed features, a number around 10 works well. If desired, a principled way to determine the number of dimensions would be to check the loss of the final MDS stage as a function of embedding dimension. To determine k for k-means we recommend constructing a 2D PHATE plot of the data to visually inspect it to determine a baseline number of clusters needed. As shown in Appendix A.1 Local CorEx doesn't suffer greatly from smaller sample sizes so long as there are at least 50 to 100 samples in the cluster. For determining the number of CorEx factors to be learned, we can examine the total correlation explained by each factor and only keep factors where the contribution isn't nominal. For thresholding learned HOIs, we discovered that visualizing them where you can see the mutual information context is helpful in determining where to stop. One could construct a scree plot and determine a threshold in that manner if desired.

2.3 Applications

2.3.1 Ablation Study - Synthetic Data

To demonstrate the effectiveness of Local CorEx to learn HOIs we constructed a synthetic dataset containing two clusters where the variable interactions are known and consist of grouped pairwise interactions, since to the best of our knowledge no standard dataset with known HOIs exists. We also compare to standard Linear and Bio CorEx. In this ablation study, we show how all methods perform when we vary the number of chosen latent factors, the difference in interactions between partitions, and the number of data samples. For each simulation setup, we ran 16 replicates.

The synthetic dataset is composed of two clusters sampled respectively from two multivariate normal distributions following two setups. In each setup, we varied the class means between the two distributions using the parameter α to control the degree of separation ($\alpha = 0$ when clusters share the same mean and $\alpha = 1$ when cluster means are maximally

Table 2.1: Each value listed in this table is the average value across the 16 replicates for that setup. Based on the metrics, Local CorEx outperforms the global methods when the data have mixed variable interactions. For sufficiently large sample sizes, Local CorEx also outperforms or approximately matches the global methods when interactions are not mixed.

			Group				Top K Latent Factor			
			Cosine Dist		AUCPRC		Cosine Dist		AUCPRC	
Data	Size	Method	$\alpha = 0.0$	$\alpha = 1.0$	$\alpha = 0.0$	$\alpha = 1.0$	$\alpha = 0.0$	$\alpha = 1.0$	$\alpha = 0.0$	$\alpha = 1.0$
Disjoint	10	Linear	0.514	0.489	**0.529**	0.505	**0.464**	0.451	**0.581**	0.548
		Bio	0.541	0.455	0.475	0.488	0.500	0.454	0.522	0.545
		Local Lin	0.539	0.413	0.477	**0.663**	0.481	**0.352**	0.517	**0.730**
		Local Bio	**0.359**	**0.394**	0.515	0.557	0.485	0.434	0.523	0.600
	100	Linear	0.484	0.424	0.607	0.591	0.406	0.403	0.702	0.627
		Bio	**0.447**	0.473	0.607	0.509	**0.351**	0.436	**0.716**	0.602
		Local Lin	0.477	**0.190**	**0.610**	**0.894**	0.408	0.086	0.687	**0.989**
		Local Bio	0.473	0.226	0.563	0.841	0.388	**0.080**	0.654	0.980
	1000	Linear	0.516	0.408	0.606	0.604	0.412	0.401	0.701	0.640
		Bio	**0.414**	0.474	**0.652**	0.507	**0.323**	0.436	**0.758**	0.617
		Local Lin	0.504	**0.146**	0.607	**0.902**	0.420	0.031	0.691	0.999
		Local Bio	0.451	0.163	0.610	0.883	0.374	**0.028**	0.708	**1.000**
	10000	Linear	0.526	0.412	0.599	0.598	0.411	0.408	0.696	0.633
		Local Lin	**0.140**	**0.142**	**0.907**	**0.907**	**0.024**	**0.026**	**1.000**	**1.000**
Non-disjoint	10	Linear	**0.354**	**0.255**	**0.739**	**0.801**	**0.278**	**0.197**	**0.828**	**0.871**
		Bio	0.401	0.312	0.634	0.670	0.308	0.290	0.751	0.763
		Local Lin	0.453	0.385	0.626	0.712	0.402	0.341	0.697	0.770
		Local Bio	0.371	0.377	0.569	0.585	0.443	0.410	0.624	0.638
	100	Linear	**0.207**	**0.197**	**0.883**	0.856	**0.107**	0.103	**0.989**	0.952
		Bio	0.236	0.289	0.842	0.754	**0.107**	0.206	0.981	0.888
		Local Lin	0.254	**0.197**	0.844	**0.890**	0.163	0.106	0.934	**0.987**
		Local Bio	0.289	0.239	0.782	0.827	0.157	**0.088**	0.920	0.979
	1000	Linear	**0.191**	0.182	**0.886**	0.856	0.091	0.086	**0.992**	0.959
		Bio	0.203	0.276	0.869	0.773	**0.084**	0.199	0.991	0.892
		Local Lin	0.197	**0.141**	0.870	**0.908**	0.100	0.051	0.959	**1.000**
		Local Bio	0.222	0.173	0.850	0.873	0.107	**0.039**	0.965	**1.000**
	10000	Linear	**0.189**	0.183	**0.887**	0.855	**0.089**	0.086	**0.993**	0.958
		Local Lin	0.190	**0.139**	0.883	**0.912**	0.090	**0.048**	**0.993**	**1.000**

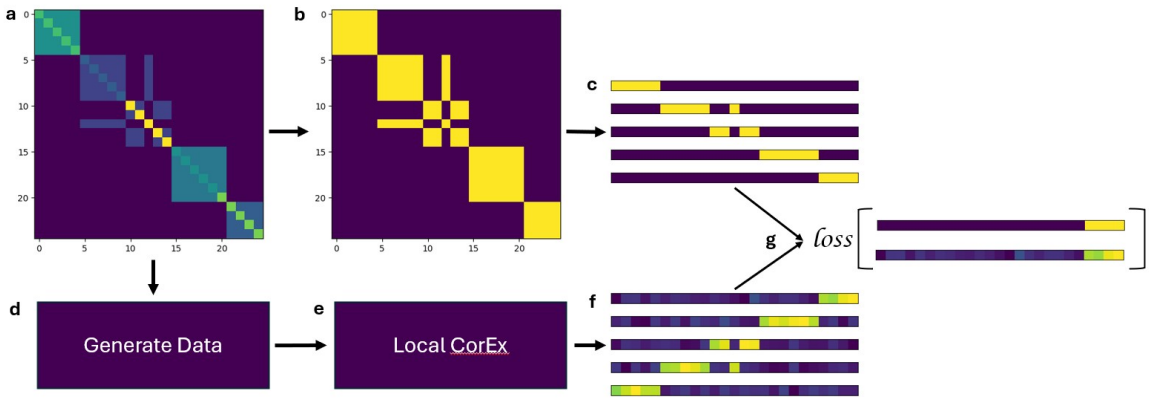


Fig. 2.2: Overview of how scoring was conducted for synthetic data. *(a)* Image of one of the covariance matrices used for generating a cluster. *(b)* All non-zero entries in each row or column are considered to be part of the local HOI. *(c)* We isolate unique ground truth HOIs. *(d)* We generate data using the covariance matrix. *e* We run Local CorEx to partition and learn the HOI. *f* We examine MI between each CorEx factor and the original features and set them as the learned HOIs. *g* We compute the cosine distance and precision under the recall curve between the learned HOIs and the ground truth HOIs.

separated). This controlled how close the two clusters are in proximity and as a result how pure the clusters are after partitioning. In the first setup, which we call non-disjoint, the two clusters share the majority of their interactions. In the second setup, which we label as disjoint, none of the interactions match. The interactions are identified by extracting each row from the covariance matrix and then examining all of the nonzero values in the row as shown in part b of Figure 2.2. To find the set of interactions present in the data we collect all of the unique rows and store them in a set (See part c of Figure 2.2. See Appendix A.1 for additional details about synthetic data generation.)

To find each predicted HOI we first ran either Linear CorEx or Bio CorEx on all, or a portion, of the simulation data points as shown in Figure 2.1. Then, for each CorEx latent factor, we examined the mutual information between the learned latent factor and the original covariates in the data (See part f of Figure 2.2). To score how well the predicted HOI matches the ground truth interactions, we used two metrics: 1) the cosine distance and 2) the area under the precision-recall curve (See part g of Figure 2.2). For each of these metrics, we compute two versions. The group version can be viewed from a recall perspective where we average the metrics across all of the interactions with the most similar predicted

HOIs (See Algorithm 1). The second version we refer to as "top k" is akin to precision. If there are g HOIs and l learned CorEx factors then $k = g$ if $l > g$. Otherwise $k = l$. In this case, we look at the top k CorEx factors and compare them with the closest true HOIs (See Algorithm 2). The scores per cluster are shown in Table 2.1.

From Table 2.1 we note that global methods, like Linear CorEx and Bio CorEx, suffer when data with mixed variable interactions are added. Partitioning the data prior to running these algorithms can help substantially when "good" partitions are chosen. Additionally, partitioning when the interactions are mostly the same has little impact on performance except when the sample size is sufficiently small, as we are effectively decreasing the already small sample size. This suggests that applying Local CorEx is a low-risk high reward trade-off so long as the sample size is sufficiently large for CorEx to learn appropriate latent factors. In general, Linear CorEx matches or outperforms Bio CorEx despite Bio CorEx being a more flexible method. This may be because we are using multivariate normal distributions, but it is convenient given that Linear CorEx runs much faster than Bio CorEx as shown in Figure 2.3. For the remainder of the paper, we use a local partitioning of the data and apply Linear CorEx, and refer to this as Local CorEx. Most of the time it takes to run Local CorEx is spent partitioning the data. Due to the landmarking approach implemented in PHATE (which is automatically used once the number of samples is sufficiently large), the time it takes to cluster becomes more or less constant. This allows Local CorEx to scale well to large datasets.

Finally, in Figure 2.3 we see that Linear CorEx is faster than Bio CorEx.

2.3.2 Communities Dataset

To demonstrate Local CorEx on real data, we used the communities and crime dataset [1] from the UC Irvine Machine Learning Repository, which consists of aggregated community data for 1,994 different communities spread across the United States. The data consists of 128 features from socio-economic data extracted from the 1990 US Census, law enforcement data extracted from the 1990 US LEMAS survey, and crime data extracted from the 1995 FBI UCR.

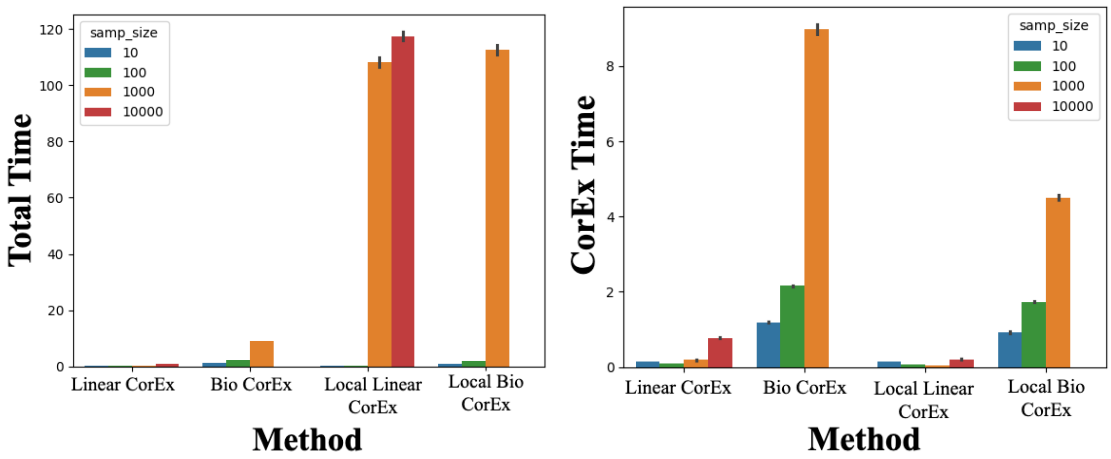


Fig. 2.3: Computational comparison of Local CorEx to Bio and Linear CorEx. The average time in seconds to complete the 16 simulations is calculated for each setup. On the left side, we include the clustering time, and on the right, we exclude the clustering time. Note that the majority of the computation time needed to run Local Linear CorEx is spent on clustering. In this work, we have chosen the clustering scheme as a set hyperparameter and have chosen not to explore other clustering schemes and leave this for further research. Of note, the clustering scheme more or less maxes out its time due to the landmark approach that PHATE uses after a sufficient number of samples are present. This is why despite the timing increases dramatically from a sample of size 100 to 1000 we don't see a similar increase from 1000 to 10000 (which is when the landmark approach kicks in by default).

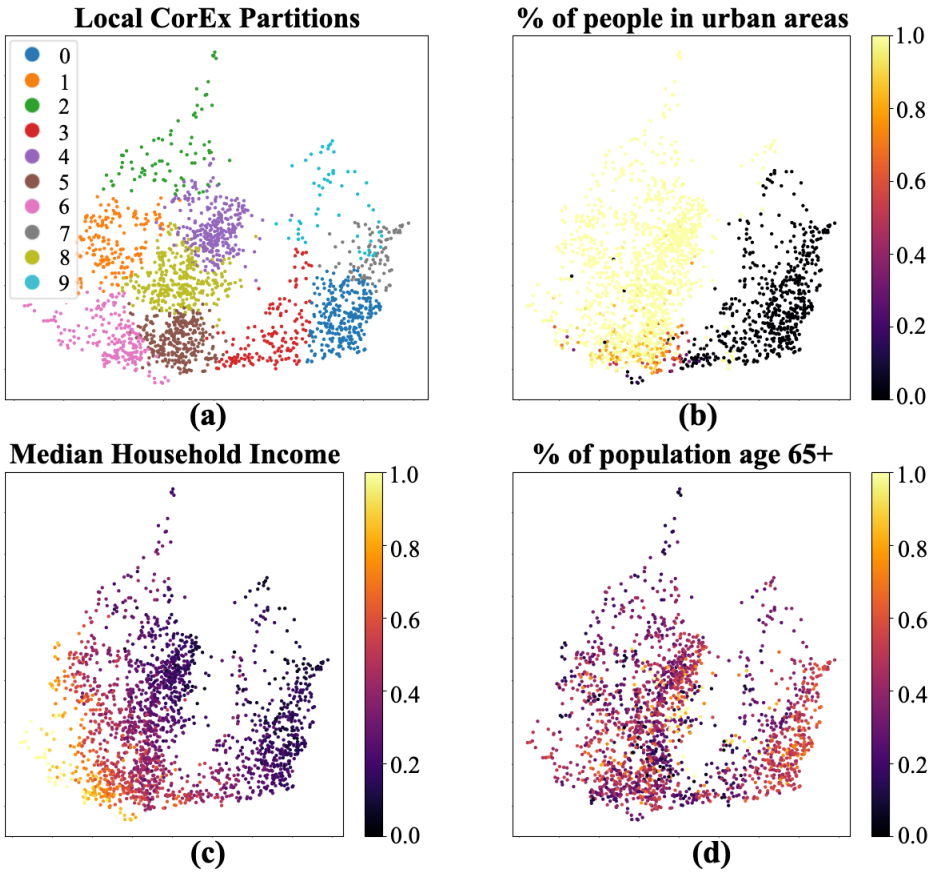


Fig. 2.4: *(a)* Two-dimensional PHATE embedding of the communities and crime dataset [1] with the 10 clusters resulting from Local CorEx plotted in different colors. *(b)* PHATE embedding with the % of people in urban areas colored. *(c)* PHATE embedding with the median household income colored. *(d)* PHATE embedding with the % of the pop age 65+ colored.

We remove all columns that contain missing values. Additionally, before partitioning the data we remove the community name, state, and cross-validation fold number columns leaving us with a total of 100 features. Next, we create a 12-dimensional PHATE embedding of the data and use k-means clustering with $k = 10$. A 2D PHATE embedding of the chosen partitions is given in Figure 2.4 as well as other PHATE plots colored by the percentage of people living in urban areas, the median household income, and the percentage of the population that is 65 or older. All variables are standardized between 0 and 1 even when some of the variable names imply a percentage. These plots show that the PHATE embedding captures meaningful structure in the data. The big cluster on the left is generally urban while the smaller cluster on the right is mostly rural. The median income increases the closer you get to the bottom left corner of the plot, and for each cluster, the lower right portion has some of the older populations. For this analysis, we further analyze clusters 0 and 8.

Cluster 0 contains data for 282 communities. The top 5 states where these communities are based are Texas, Ohio, Oklahoma, Missouri, and New York with 30, 28, 18, 15, and 14 communities, respectively. From Figure 2.4 we see that this cluster is rural, on the lower end for median income, and hosts an older population. When we look at the top 8 factors and visualize which features each factor is related to in Figure 2.5 in the Appendix, we see that the factors group variables together which make sense. For example, The first factor relates the values of unmarried young people with living in the same city since 1985. The second factor groups the different measurements for rent prices and owner-occupied housing variables. The third factor relates the total percentage of the population that is divorced with the specific values for males and females as well as being somewhat correlated with the percentage of kids in family housing with two parents.

Cluster 8 contains data for 327 communities. The top 5 states associated with its communities are Florida, New Jersey, Massachusetts, Texas, and Washington with 56, 33, 19, 19, and 18 communities, respectively. Looking at Figure 2.4 we see that this cluster is very urban, on the lower to middle end for median income, and is not consistent for

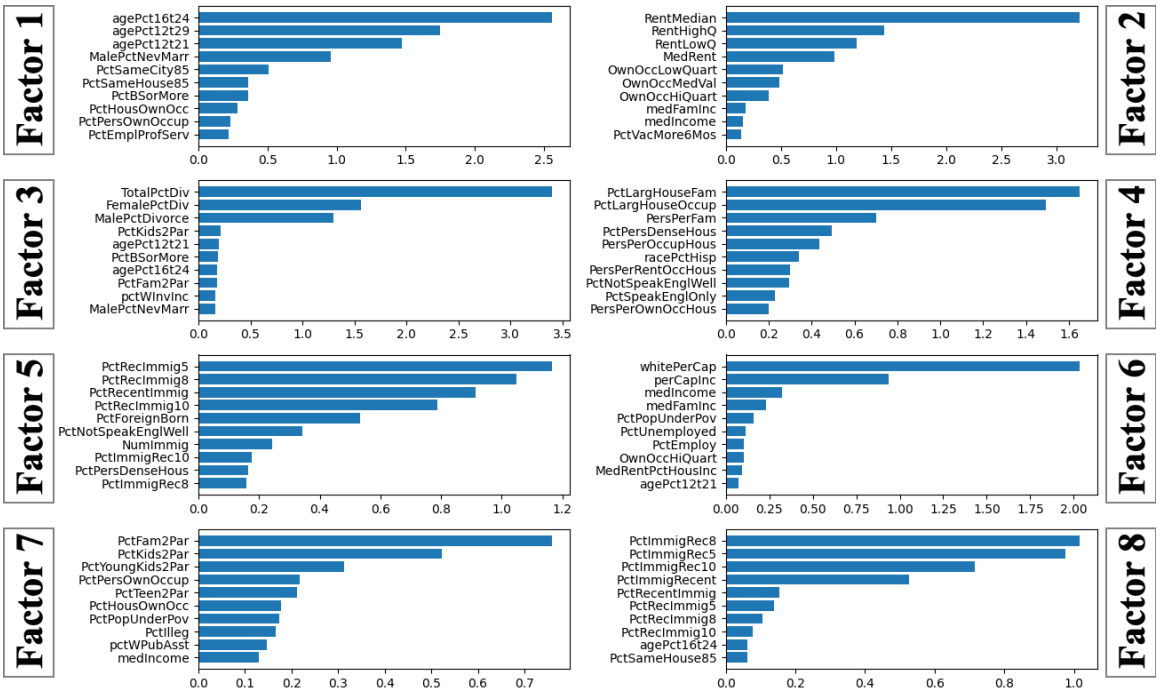


Fig. 2.5: The first 8 Local CorEx Cluster 0 latent factors from the communities and crime dataset [1] showing the mutual information between the factor and the original features sorted in descending order. This cluster consists of communities that on average are rural, on the lower end for median income, and hosts an older population. See https://archive.ics.uci.edu/dataset/183/communities+and+crime for descriptions of variable names.

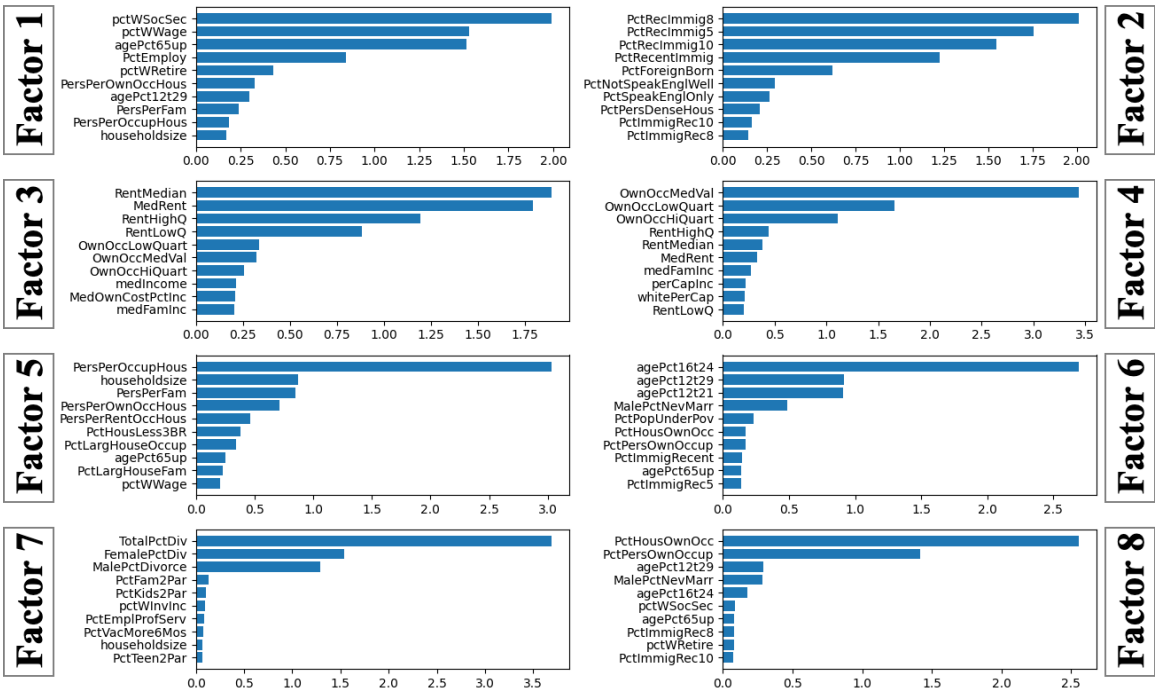


Fig. 2.6: The first 8 Local CorEx Cluster 8 latent factors from the communities and crime dataset [1] showing the mutual information between the factor and the original features sorted in descending order. This cluster consists of communities that on average are urban, on the lower to middle end for median income, and is not consistent for age above 65. See https://archive.ics.uci.edu/dataset/183/communities+and+crime for descriptions of variable names.

age above 65. We plot the mutual information between the top 8 factors with the original features in Figure 2.6, which have substantial overlap with those found in cluster 0.

We believe that this is because these groups are similar to the non-disjoint data in the ablation study. That is, the overall HOIs do not change much between groups in this setting, but the semantics of what is correlated will change between groups. For instance, latent factor 1 of cluster 0 and latent factor 6 of cluster 8 overlap substantially. They both have high mutual information with agePct16t24, agePct12t29, agePct12t21, and MalePctNevMarr. What is informative is when we examine the other variables related to these factors. For cluster 0, this factor also correlates with PctSameCity85 (percent of people living in the same city as in 1985, 5 years before), PctSameHouse85 (percent of people living in the same house as in 1985, 5 years before), PctBSorMore (percentage of

people 25 and over with a bachelors degree or higher education), PctHousOwnOcc (percent of households owner occupied), PctPersOwnOccup (percent of people in owner-occupied households), and PctEmplProfServ (percentage of people 16 and over who are employed in professional services). This is informing us about the population of these young people. They are living in the same home and often the same city, in a home where the owner lives, and are working jobs in professional services.

Cluster 8's latent variables show a different picture. Factor 6 in this group is also associated with PctPopUnderPov (percentage of people under the poverty level), PctHousOwnOcc (percent of households owner-occupied), PctPersOwnOccup (percent of people in owner-occupied households), PctImmigRecent (percentage of immigrants who immigrated within last 3 years), agePct65up (percentage of population that is 65 and over in age), and PctImmigRec5 (percentage of immigrants who immigrated within last 5 years). Thus this cluster seems to be comprised of young people who are likely living under the poverty line, renting, and have recently immigrated to the US in the last 5 years.

By using Local CorEx we were able to find the main HOIs. We also were able to compare similar factors between clusters to discover interesting details that separate the clusters. Other interactions could also be considered.

2.3.3 MNIST

As another demonstration, we apply Local CorEx to the MNIST dataset [36]. The MNIST dataset is a common Machine Learning benchmark dataset for image classification. It consists of $70,000$ handwritten digits with 28×28 pixel images of centered digits.

We first partition the data by flattening the images into vectors of size 784 and then creating a 10-dimensional PHATE embedding for the flattened data. We ran k-means clustering with $k = 24$. The resulting clusters are given in Figure 2.7.

We demonstrate Local CorEx using cluster 13, which has the following membership: 1898 9s, 1008 4s, 40 7s, 31 1s, 12 8s, and fewer than 10 each of the remaining digits. The square-rooted mutual information between each Local CorEx factor and the original features is shown in Figure 2.8. Despite the data being extremely non-linear, Local CorEx

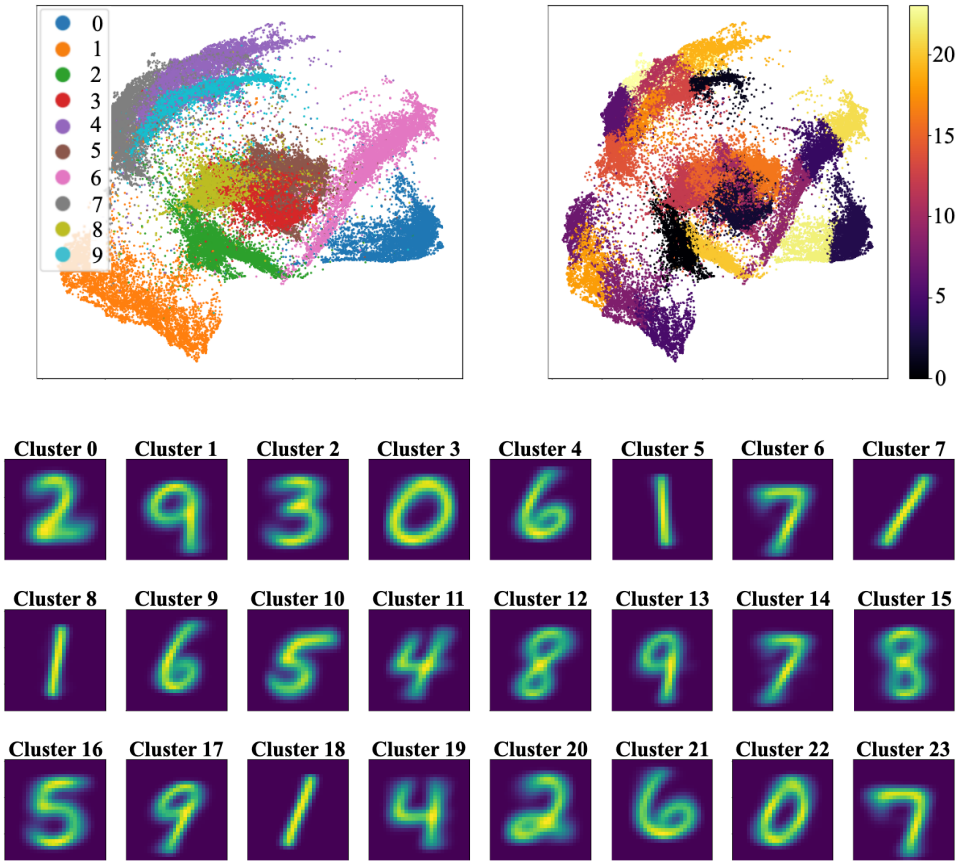


Fig. 2.7: *(Top left)* A two-dimensional PHATE embedding of the MNIST dataset colored by data labels. *(Top right)* The same 2D PHATE embedding colored by the Local CorEx clusters. The clusters largely respect the boundaries between classes. *(Bottom)* The average pixel values found in each of the clusters. The outline of the majority digit present in each cluster is easily visible.

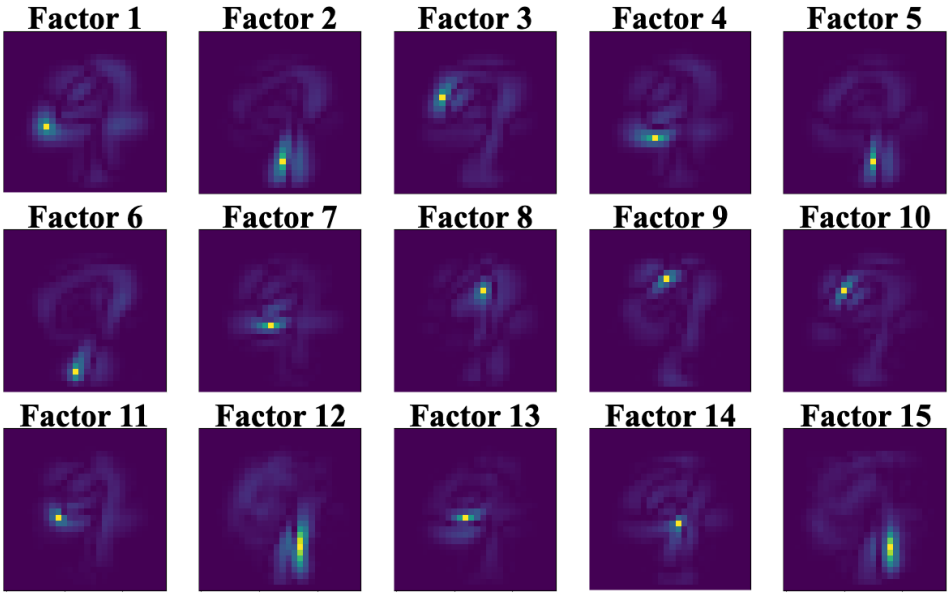


Fig. 2.8: Visualizing the square-rooted mutual information between the first 15 Local CorEx factors trained on cluster 13 with the original features of the MNIST dataset.

is able to capture HOIs in the data that make sense. For example, factor 1 captures the far point where a four would likely bend while factor 2 captures the stem of the fours or nines. Other factors similarly appear to be related to constructing either fours or nines. While Local CorEx can be viewed as a piece-wise linear method, these results suggest that this is sufficient to capture many non-linearities. See Appendix A.2 for additional examples of Local CorEx clusters.

2.3.4 Neural Network Model Interpretability

Our goal is to uncover how information is structured and shared within neural network layers. We tackle this in three steps on MNIST cluster 16 in Figure 2.7:

1. **Redundancy analysis.** Measure how dropout affects total correlation across layers.

2. **Feature visualization.** Identify cohesive node-groups via Local CorEx and decode their role by reconstructing perturbed activations or running the deep dream algorithm to generate optimized samples.

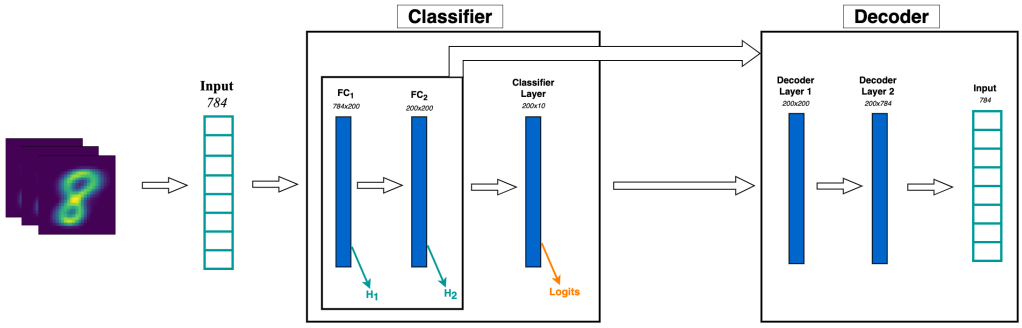


Fig. 2.9: Neural Network architecture used in Section 2.3.4. MNIST digit samples are first flattened into vectors of size 784 before being passed into the neural network. As the inputs are passed through each layer in the classifier we save each internal representation. After a classifier is fully trained we also train a decoder model that uses the first layers of the classifier as an encoder whose weights are frozen during the decoder's training.

3. **Performance impact.** Delete those node-groups and measure accuracy across clusters.

After these MNIST experiments, we extend our framework to a ResNet-18 on ImageNet (Section 2.3.5).

For our MNIST model case study, we use Local CorEx to explore the hidden representations and model weights for a neural network classifier trained on the MNIST dataset. After the classifier is fully trained, we train a decoder that is used to visualize reconstructed perturbed hidden layer representations. The full model architecture is in Figure 2.9. All explorations are conducted on the test data which is partitioned into 20 clusters using k-means clustering on a 10-dimensional PHATE embedding of the data. The average pixel value in each cluster is visualized in Figure 2.10. For the following analyses, we focus on cluster 16 for brevity but analyze an additional cluster in Appendix A.3. Cluster 16 was chosen due to the heterogeneity of the class labels between 9s and 4s and its similarities to cluster 13 in Section 2.3.3.

Effect of Dropout

It is assumed that neural networks trained with dropout [37] tend to duplicate the passing of information across layers, so when a node is dropped the information is still

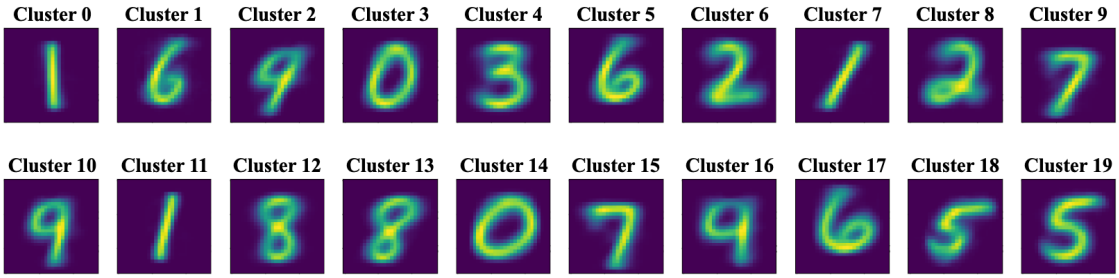


Fig. 2.10: Visualization of the average samples in each of the 20 partitions for the MNIST test dataset used in Section 2.3.4.

passed forward. If this is true, then the duplication of information passed would also increase the total correlation present in a layer. To explore this, we trained a series of paired classifier models on the MNIST dataset. All models have two hidden layers and share all hyperparameters except, in each pair, one uses dropout with 0.5 and the other does not use dropout. To help simplify the comparison, each pair of models uses the same initialized set of weights.

We feed the data from cluster 16 into the trained classifiers, where the output of each layer is viewed separately as a hidden state representation of the data. We then run Linear CorEx on each of the hidden states (i.e. layer outputs) separately, measure the total correlation explained by each latent factor, and report the summed total correlation values for the first 5 latent factors. These total correlation measurements across hidden layers should measure the total amount of information shared among the hidden nodes and be a good surrogate for how much redundancy of information is present in a hidden layer. We repeated this study on pairs of classifier models with 3 hidden layers to further explore the effect of deeper networks. Table 2.2 shows the average values obtained across the 10 pairs of models for both 2 and 3 hidden layers.

In all scenarios, the total correlation in the first hidden layer is slightly smaller for the models with dropout. This is in stark contrast to what we see for the later layers where the amount of total correlation is substantially larger in the models with dropout compared to the models without. Our hypothesis for this is that the first layer extracts basic features from the data, such as relevant clusters of pixels. These features are by nature sparse and

Table 2.2: Average Total Correlation values of the mutual information between the first 5 CorEx factors and the original features. Total correlation increases as models become deeper both with and without dropout. Dropout exacerbates this trend in the later layers.

	Total Correlation	
Layer	no dropout	with dropout
2 layer H1	31.2 ± 1.6	27.2 ± 2.0
2 layer H2	51.9 ± 2.8	98.1 ± 4.4
3 layer H1	33.7 ± 2.5	26.4 ± 1.2
3 layer H2	40.8 ± 2.8	95.9 ± 3.2
3 layer H3	76.1 ± 4.6	162.4 ± 8.5

likely do not benefit much from the dropout push to become less sparse. In later layers, the model has had the chance to learn useful features that it can distribute across multiple nodes to ensure that the feature is preserved if the node is dropped. This suggests that the benefits of dropout may not be equal in different layers within the neural network.

Visualizing hidden node effects

Having seen that dropout reshapes the information landscape within layers, we next ask which sets of nodes actually carry class-defining features. To answer this, we pair hidden-layer activations with their corresponding logits and run Local CorEx on the concatenated vectors, allowing us to identify cohesive groups of nodes and associate each group with specific class signals (see Figure 2.11). The logits are included to aid with interpretability. To visualize the impact of these node groups, we then train a decoder—using the first two layers of the classifier as a frozen encoder—to reconstruct inputs from the perturbed hidden representations, thereby revealing how each set of related nodes contributes to the network's learned features.

Using this setup, we fed all samples from cluster 16 into the network. The corresponding outputs of the first hidden layer give a new 200-dimensional representation of the data. We then averaged these representations and perturbed the coordinates corresponding to the hidden nodes found by the Local CorEx factor in proportion to their mutual information with the hidden nodes. We then decoded the resulting perturbed first hidden layer repre-

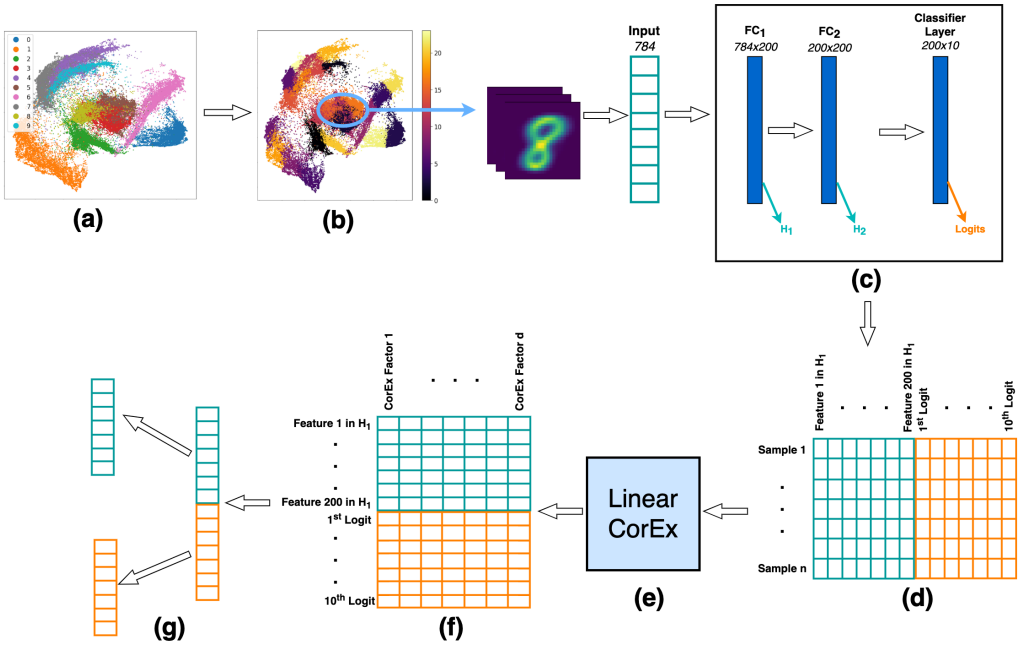


Fig. 2.11: Running Local CorEx on a hidden representation of the inputs concatenated with their respective logits. *(a)* PHATE embedding of the MNIST data, colored by class. *(b)* k-means clustering is applied to the PHATE embedding to generate the partition. *(c)* MNIST digit samples are flattened into vectors of size 784 before being passed into the neural network. As the inputs from the cluster are passed through each layer in the classifier we save each internal representation. *(d-e)* The hidden layer outputs (e.g. H_1 hidden state) are concatenated with their respective logits and passed through Linear CorEx. We repeat this for the hidden state H_2 with their respective logits. *(f)* We visualize the mutual information between the learned CorEx factors and the concatenated features with their respective logit. *(g)* We pick one latent factor to continue to explore and split back into the concatenated feature vector and its respective logits. This allows us to transform hidden state representations into reconstructed inputs and identify groups of hidden nodes to delete.

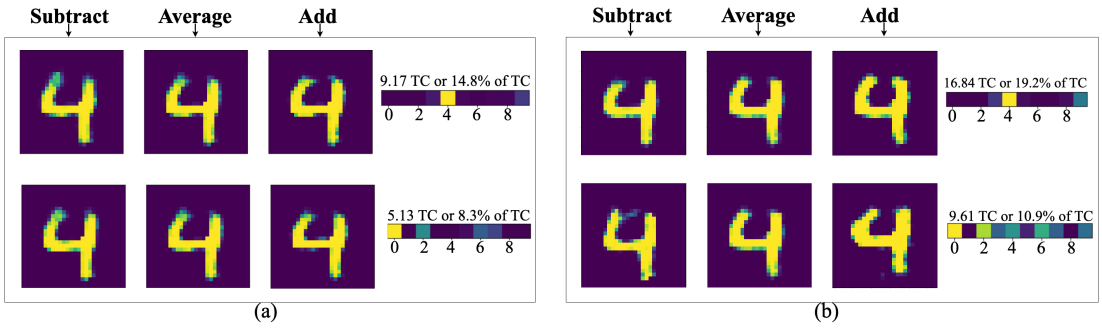


Fig. 2.12: Visualizing the effect of perturbing the average neural network hidden state representations of cluster 16 in the MNIST test dataset. *(a)* The plots are associated with perturbing the H1 representation. This first row is associated with the first Local CorEx factor and the second row is associated with the second Local CorEx factor. *(b)* Same as in *(a)* but for H2. For each group of plots, the leftmost column image is generated by subtracting the mutual information between the Local CorEx factor and the hidden nodes from the average representation. The second column image gives the average hidden state representation. The third column image is generated by adding the mutual information between the Local CorEx factor and the hidden nodes from the average representation. Finally, the rightmost column plots the mutual information between the Local CorEx factor and the model logits. This analysis gives us a visual intuition for what role the grouped hidden nodes play.

sentation. The results can be seen for the first 2 latent factors on the left side of Figure 2.12. Perturbing the values of the hidden nodes associated with the first latent factor alters the curvature of the digit in the top left portion of the number. Perturbing the hidden nodes associated with the second Local CorEx factor removes pixels on the base of the four, the base of the horizontal line in the center, and the top part of the number. Removing the pixels in these locations is associated with the probability of the sample being classified as a zero or a two, as indicated by the mutual information.

We repeat this for the second hidden layer, with the results shown on the right side of Figure 2.12. Here we see very similar effects as that of perturbing the first hidden layer. Perturbing the hidden nodes associated with the first Local CorEx factor changes how the pixels come together or not near the top of the four. This factor is mostly associated with the 4 and 9 logits. The second Local CorEx factor affects the digit's thickness, the location of the middle bar, the angle of the top left line, and where the base of the stem begins. This factor is associated with various logits that will be influenced by these pixels.

Table 2.3: Baseline classifier accuracies across clusters on the test split of the MNIST dataset. With the accuracies, for each cluster, we include the number of each label for the 3 most represented digits in the cluster. From this we can see that the baseline model performs well across clusters.

Clusters	Accuracy	Cluster Composition - Top 3
0	98.60	1: 404, 4: 9, 7: 8
1	96.57	6: 200, 5: 11, 2: 6
2	98.08	4: 314, 9: 249, 8: 5
3	98.15	0: 406, 2: 11, 5: 6
4	98.20	3: 799, 5: 172, 8: 78
5	99.23	6: 384, 4: 3, 0: 1
6	97.80	2: 527, 7: 7, 1: 5
7	97.49	1: 338, 7: 9, 2: 5
8	99.54	2: 433, 9: 1
9	97.07	7: 461, 9: 22, 2: 20
10	97.86	9: 475, 4: 324, 7: 29
11	99.49	1: 378, 7: 7, 4: 4
12	98.00	8: 468, 3: 177, 5: 14
13	99.51	8: 379, 9: 12, 3: 8
14	99.64	0: 557, 2: 2, 6: 2
15	98.26	7: 496, 9: 7, 3: 6
16	98.88	4: 317, 9: 212, 7: 4
17	99.17	6: 359, 4: 2, 9: 1
18	97.78	5: 247, 8: 11, 0: 4
19	98.90	5: 436, 3: 8, 8: 6

Quantifying hidden node effect on classifier performance

In the previous section, we visually inspected the effect of perturbing hidden nodes associated with a Local CorEx factor. Here we delete the hidden nodes in the classifier model and measure their impact on the classifier's performance across clusters to determine if the related nodes have a local or global impact on the classifier's performance. We start by obtaining a baseline to see how the classifier performs across clusters, with the results shown in Table 2.3. The model performs relatively well across all clusters. We then use the same Local CorEx factors learned in the Visualizing hidden node effects subsection in Section 2.3.4 and create reduced models by deleting the hidden nodes associated with each of the factors. We then recompute the accuracies across all clusters using the reduced model.

We start with the Local CorEx factors learned from the H1 embeddings concatenated

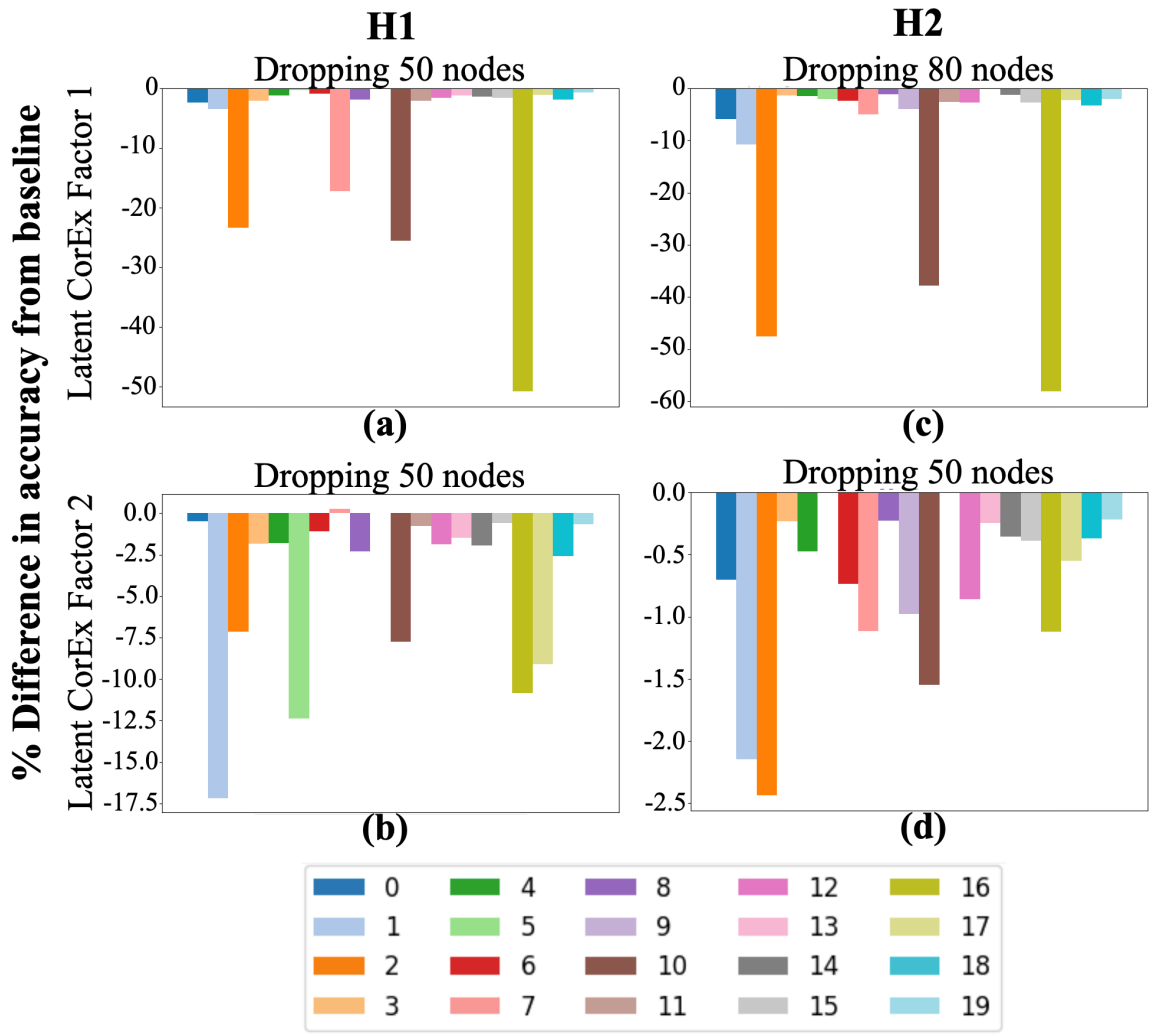


Fig. 2.13: *(a)* Difference in classification accuracy between the unaltered model and the model after deleting 50 hidden nodes in the first hidden layer with the highest mutual information with the first Local CorEx factor associated with cluster 16. *(*b) Same as *(a)* except when using the second CorEx factor to determine the 50 hidden nodes to delete. *(*c) Same as in *(a)* except when deleting 80 hidden nodes in the second hidden layer. *(*d) Same as in *(b)* except in the second hidden layer. Note that the y-axis scale differs for each plot.

with the logits. The first factor is associated with the logit for classifying fours. When we delete the 50 nodes with the highest mutual information associated with this factor and recompute the accuracies across clusters, it has a large impact on classification accuracy for clusters 2, 10, and 16, as shown in Figure 2.13. All of these clusters have large quantities of 4s and 9s present. Remarkably, almost all other clusters are relatively unaffected by this despite deleting 25% of the hidden nodes in the first layer. The second factor is associated with several logits including 0's, 2's, and 6's. When 25% of the nodes associated with this factor are dropped the effect is more widespread. The clusters most impacted are 1, 2, 5, 10, 16, and 17. Clusters 2, 10, and 16 contain 4s and 9s so it isn't surprising that removing a feature calculated on a group of 4s and 9s affects their classification. Clusters 1, 5, and 17 are composed almost entirely of 6s. So although this factor has high mutual information with the 0, 2, and 6 logits it affects mostly the classification accuracy of groups of 6s.

We continue our analysis with the H2 embeddings concatenated with logits. The first Local CorEx factor is associated with the logit for classifying fours. When we delete the first 80 nodes with the highest mutual information associated with it and recompute the accuracies across clusters we see it has a large impact on classification accuracy for clusters 2, 10, and 16 which contain 4s and 9s. Almost all other clusters are relatively unaffected by this despite deleting 40% of the hidden nodes in the second hidden layer. Repeating this for the second latent factor, which is associated with several logits including 0's, 2's, and 6's, seems to have little effect on the model's overall performance despite it representing 25% of the layer's hidden nodes. The clusters most affected though are 1, 2, 10, and 16, which contain mostly digits with a flat horizontal line in the center of the digit (Figure 2.10). Additionally when we look at the effect of perturbing this factor in Figure 2.12 we see that it affects the height of the middle bar. Although this feature is less important for classification, the effect on accuracy across partitions makes sense.

2.3.5 ResNet Results

While MNIST lets us probe small-scale digit features, we now investigate whether similar higher-order patterns emerge in real-world vision by analyzing a pretrained ResNet-

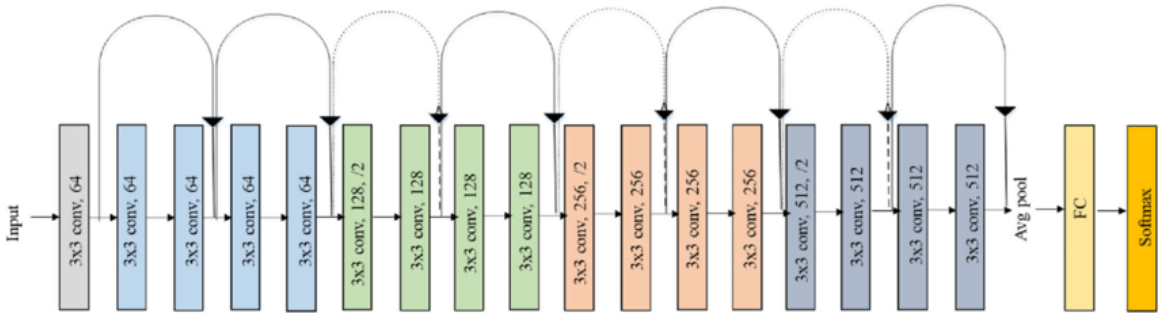


Fig. 2.14: ResNet18 model architecture [2].

18 model [38]. This network comprises four residual blocks (18 layers in total), and we examine the interactions among features in its final convolutional layer after mean pooling. Using the ImageNet validation set [39] as input, we pass each image through the model, extract its representation immediately before the final linear classifier, and embed these vectors into 10 dimensions via PHATE. Finally, we apply k-means clustering (with $k = 100$) to the PHATE embeddings and visualize the resulting clusters in a 2D PHATE plot (Figure 2.15).

From the resulting clusters, we focused on Cluster 58 (see Figure 2.16), which consists exclusively of African Grey samples. We extracted and saved both the average-pooled feature representations and the corresponding model logits for each input in this cluster. To uncover local higher-order interactions, we concatenated each average-pooled vector with its logits and then applied Linear CorEx to these augmented representations.

Cluster 58

After learning the Local CorEx components, we examine which factors have a high value of mutual information with the logit associated with African Greys, which happens to be factors 1 and 5. We can try to get an idea for what these factors do by running an altered version of deep dream [40] where instead of optimizing an input to maximally activate a single convolutional filter we try to maximally optimize a weighted set of activations where the weight is determined by the mutual information between the learned factor and the associated filters. This is shown in Figure 2.17. The first factor seems to have found the

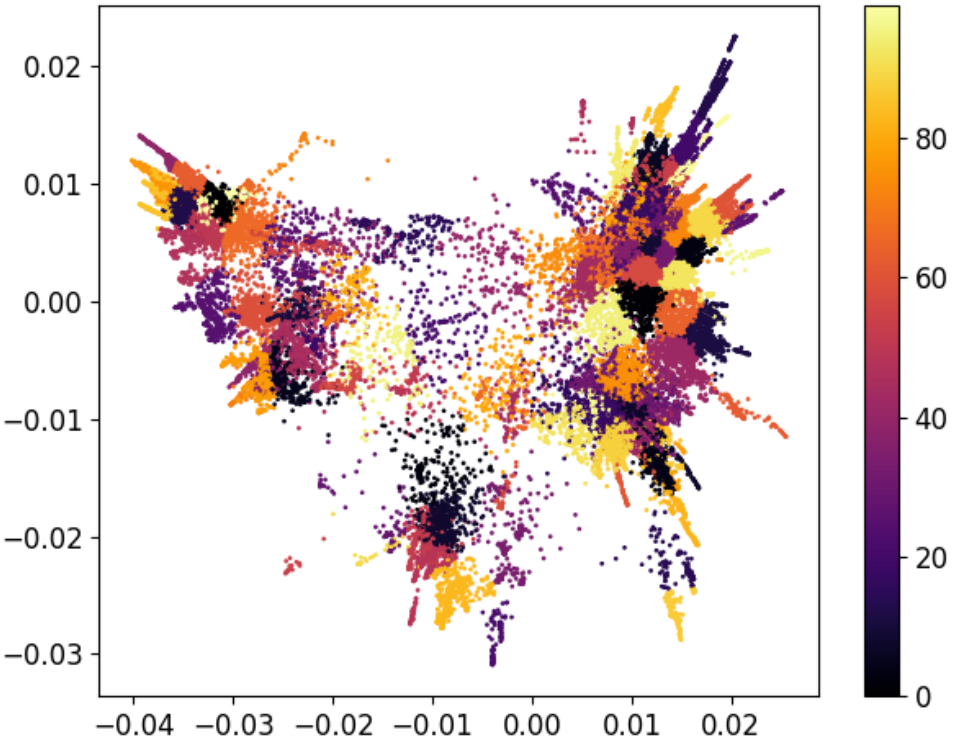


Fig. 2.15: 2D PHATE plot of the imagenet's validation datasets average pooling layer representation from a pretrained ResNet18 model. The clustered are colored by their k-means cluster assignment.

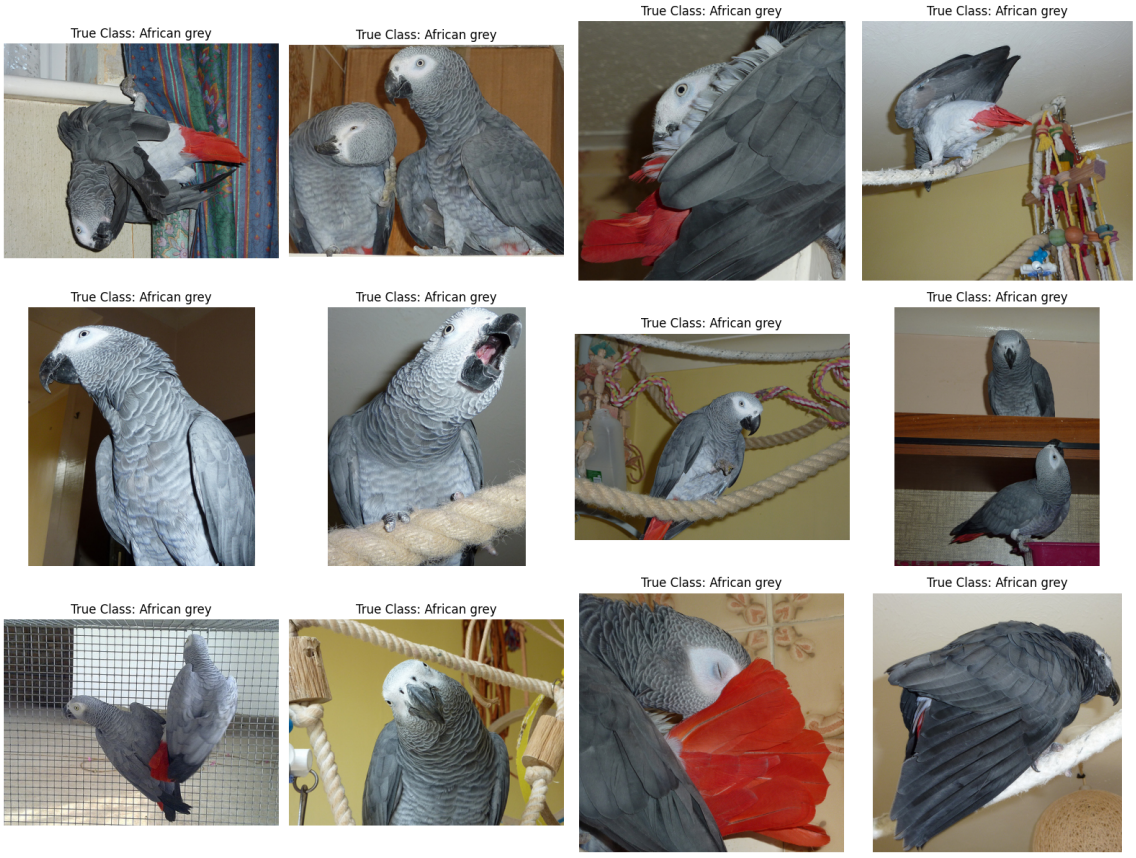


Fig. 2.16: Twelve randomly selected ImageNet validation images from Cluster 58, as identified by Local CorEx on the ResNet-18 mean-pooled convolutional features. Each panel is labeled with its true class. Cluster 58 is composed entirely of African greys.

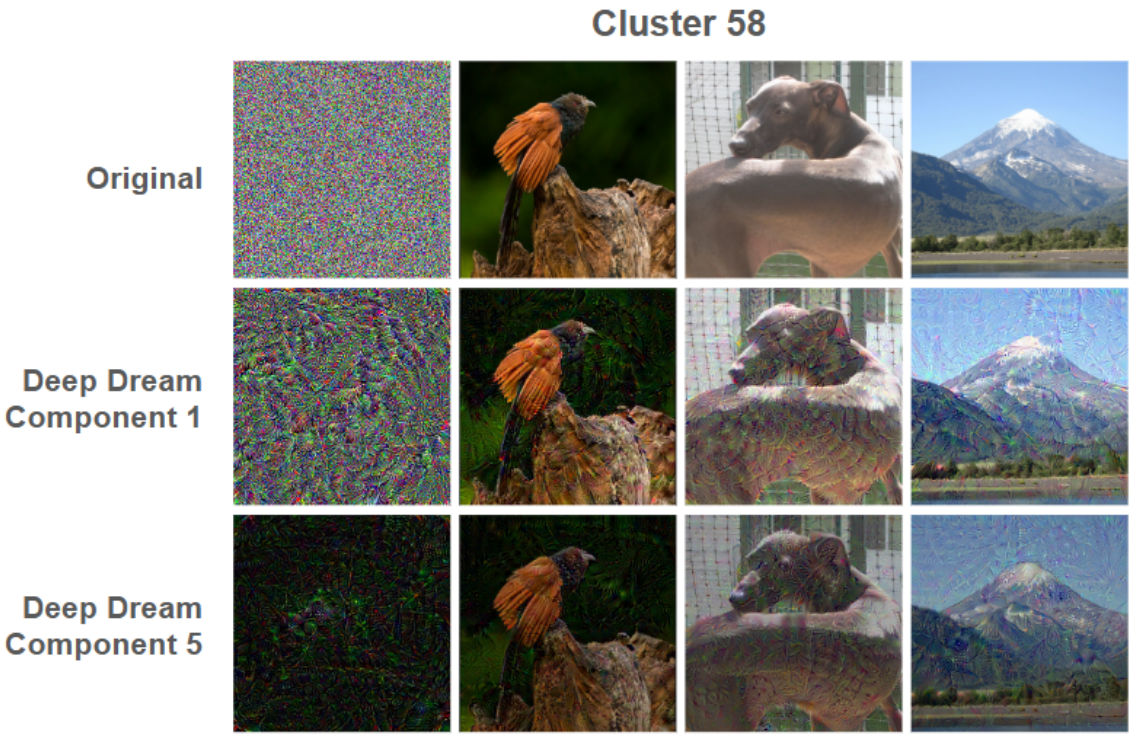


Fig. 2.17: Modified Deep Dream visualizations of Local CorEx components on a pretrained ResNet-18. *(Top row)* a random noise input (far left) and three unaltered ImageNet validation images. *(Middle row)* the same inputs after weighted gradient-ascent (Deep Dream) on the ResNet-18's mean-pooled convolutional features, with weights given by Local CorEx Factor 1. *(Bottom row)* analogous Deep Dream outputs using Local CorEx Factor 5. Both factors highlight feather-like textures

feather pattern seen on the African grey's neck, chest, and head while the 5th factor seems to have found the pattern of the shape of the white area around the African grey's eye that is white.

We can see that these attributes are key for the ResNet18 to classify the model by examining how the baseline accuracy is affected when we delete the top 100 nodes with the highest mutual information with each of these factors in Figures 2.18 and 2.19. In this same figure, we can see that this seems to exclusively affect the African grey logits.

If we look at the second most affected cluster for factor 1, we see in Figure 2.20 that this is cluster 28 and that the highest effected labels are all birds. From a brief visual inspection of some of the images present in the cluster as shown in Figure 2.21 we can postulate how

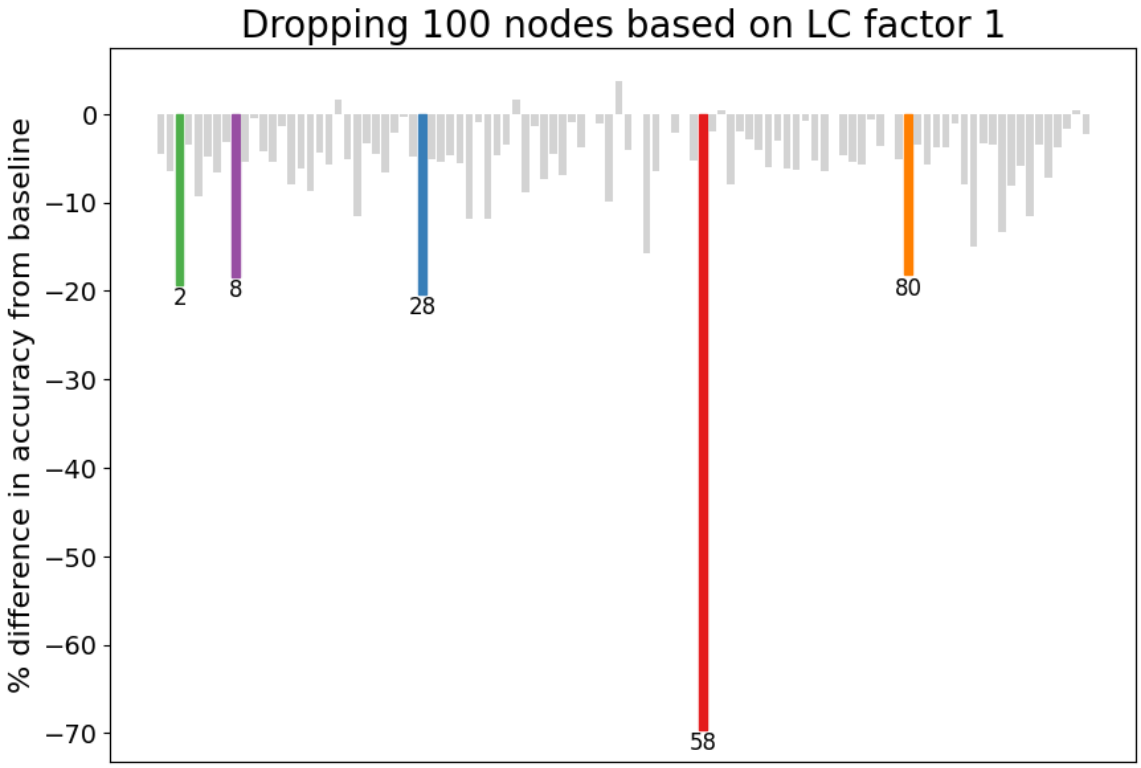


Fig. 2.18: This plot shows the difference in classification accuracy between the original ResNet model and the reduced model across all clusters. The reduced model is generated by looking at Local CorEx factor 1 when trained on cluster 58 and deleting the 100 nodes with the highest mutual information with the Local CorEx factor. Notice that the accuracy in cluster 58 is the most affected with the majority of the other clusters having much smaller impacts.

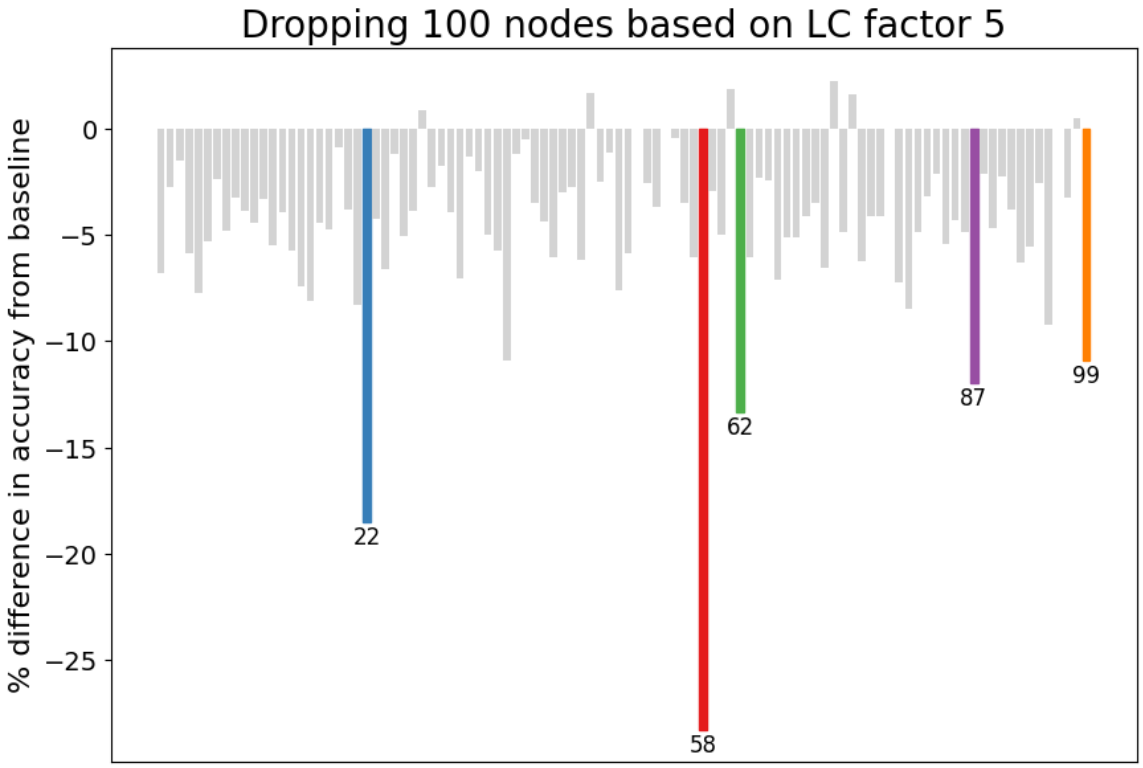


Fig. 2.19: This plot shows the difference in classification accuracy between the original ResNet model and the reduced model across all clusters. The reduced model is generated by looking at Local CorEx factor 5 when trained on cluster 58 and deleting the 100 nodes with the highest mutual information with the Local CorEx factor. Notice that the accuracy in cluster 58 is the most affected with the majority of the other clusters having much smaller impacts.

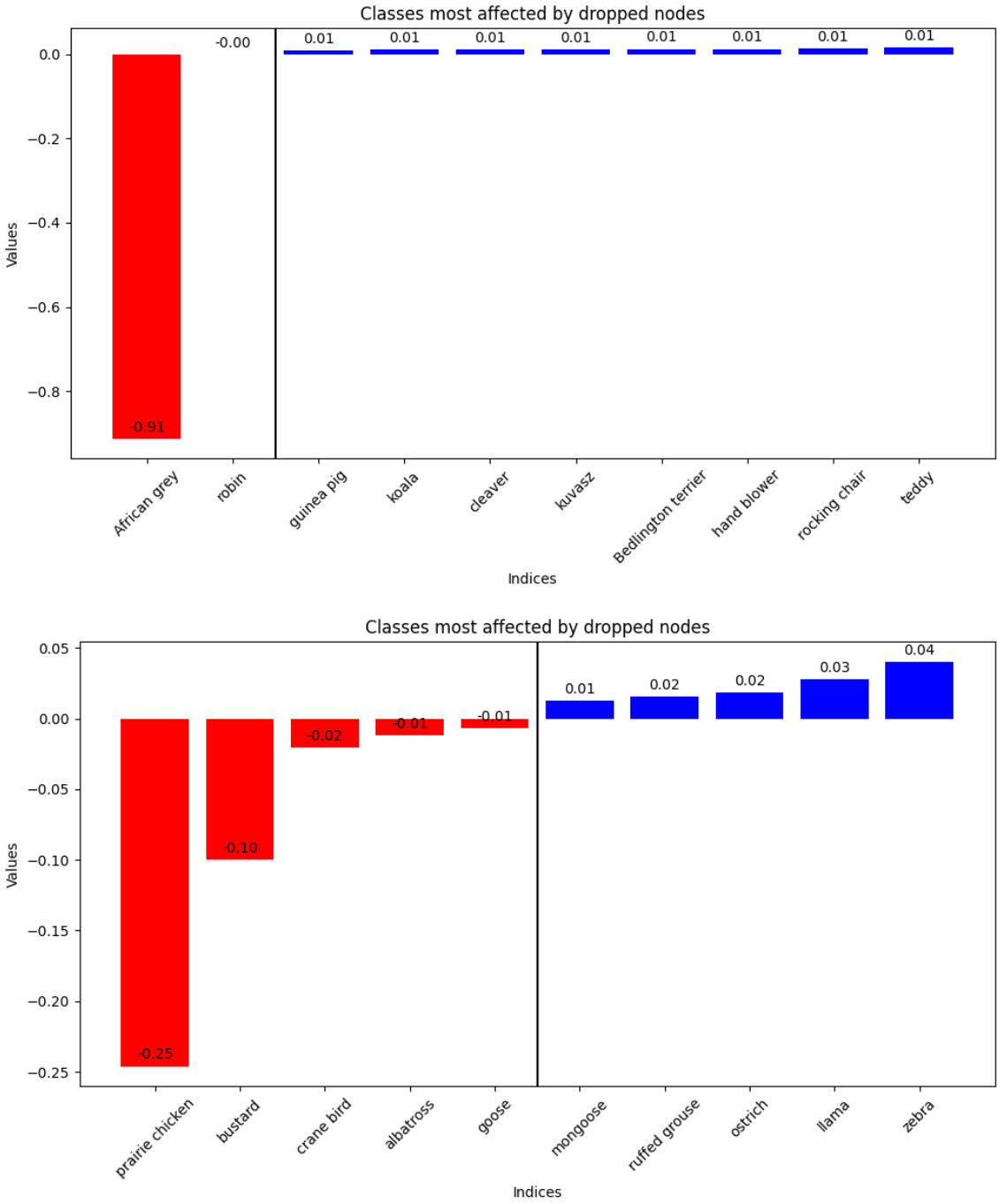


Fig. 2.20: Impact on ResNet-18 class logits after ablating the top 100 hidden-layer nodes identified by Local CorEx Factor 1 (derived from Cluster 58). In both panels, red bars (left of the vertical zero line) mark the classes whose logits decreased the most, while blue bars (right) mark the classes whose logits increased the most. **Top:** Results for Cluster 58 (primarily African Greys) — note that "African grey" is almost exclusively affected. **Bottom:** Results for Cluster 28 (the second most affected cluster, containing various bird species). Removal of feather-related activations reduces scores for "prairie chicken", "bustard", "crane bird", "albatross", and "goose" and increases them for "ostrich", "llama", and "zebra" indicating a shift from feather to stripe/fur pattern features.



Fig. 2.21: Twelve randomly selected ImageNet validation images from Cluster 28, as identified by Local CorEx on the ResNet-18 mean-pooled convolutional features. Each panel is labeled with its true class. Cluster 28 is dominated by birds.

removing the feather feature from prairie chickens might make the model think it is more likely to be a zebra since if we remove the feather texturing we are left with what looks like black and white stripes. If we remove the feather feature from a bastard we can see why the model would increase the likelihood of it being classified as a llama.

Cluster 28

If we repeat this analysis but do it for the second most impacted cluster from the 5th corex component, we see from the results in Figure 2.22 that this is cluster 22. From examining a sampling of images found in this cluster (as shown in Figure 2.23) we see that this cluster is composed mainly of hummingbirds and jacamars. These look very similar

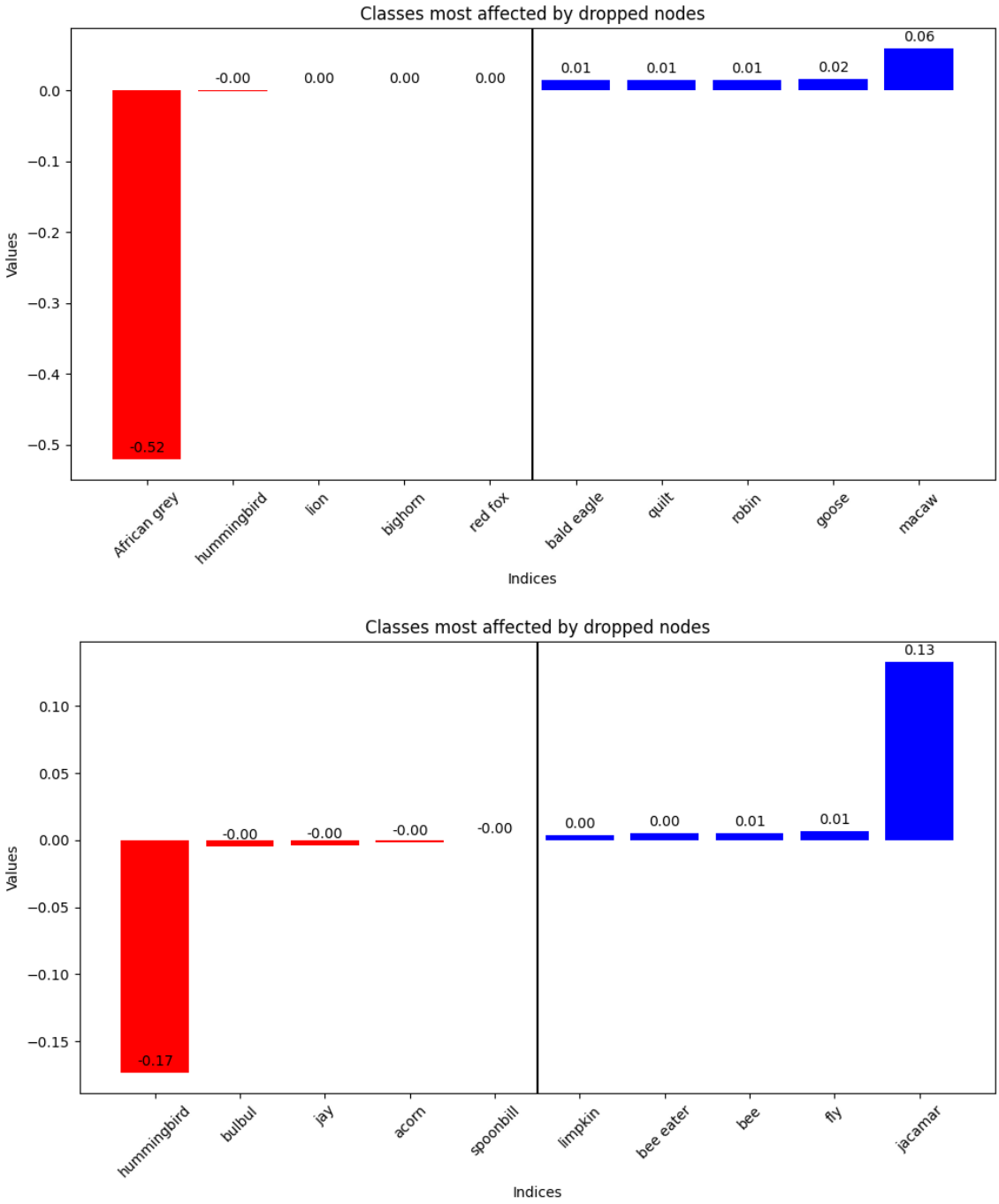


Fig. 2.22: Impact on ResNet-18 class logits after ablating the top 100 hidden-layer nodes identified by Local CorEx **Factor 5** (derived from Cluster 58). In both panels, red bars (left of the vertical zero line) mark the classes whose logits decreased the most, while blue bars (right) mark the classes whose logits increased the most. **Top:** Results for Cluster 58 (primarily African Greys). Ablating the dot-pattern feature lowers the 'African grey' logit and boosts classes with smoother plumage like "macaw" and "goose". **Bottom:** Results for Cluster 22 (the second most affected cluster, containing jacamars and hummingbirds). Removing the dot-pattern feature causes hummingbird logits to drop and jacamar logits to rise, reflecting their similar shapes but distinct feather markings.



Fig. 2.23: Twelve randomly selected ImageNet validation images from Cluster 22, as identified by Local CorEx on the ResNet-18 mean-pooled convolutional features. Each panel is labeled with its true class. Cluster 22 is dominated by Jacamars and Hummingbirds.

to one another in shape and size, but differ substantially in the patterns on their feathers, particularly on their chests. Hummingbirds seem to have patterns with dots on their chests, throats, or heads. This would make sense then that removing this feature would make these hummingbirds appear to the model to look more like jacamars.

Cluster 22

Remarkably, we can delete close to 20% of the nodes in the model layer and only impact a few classes despite the fact that there are 1000 classes. One might think that deleting 20% of the nodes would affect roughly 20% of the classes, but we don't see this effect, suggesting that these networks are very robust and that the higher-order interactions in the model are far stronger than the main effects of a single node.

2.4 Summary

We have shown that Local CorEx is superior to global methods at capturing variable interactions when these interactions vary across the data manifold. In Table 2.1, we demonstrated that even when interactions largely overlap, Local CorEx outperforms global methods with mostly pure clusters when the sample size exceeds 100 and is robust to hyperparameter selection. Furthermore, despite impure clusters, Local CorEx performs comparably to global methods once the sample size reaches 1000. The more the variable interactions vary across the data, the greater the benefit of using Local CorEx over other global methods for extracting HOIs in the data.

In this chapter, we applied Local CorEx to explore variable interactions across diverse data types, including tabular data, image data, neural network nodes, and internal representations. Using a feed-forward neural network trained on the MNIST dataset, we employed Local CorEx in an unsupervised manner to identify the hidden nodes that the model relied on for predicting a specific class. Our results demonstrated that removing these nodes selectively affected only the target class, indicating that—even amid the inherent interconnectedness of neural networks—it is possible to isolate clusters of nodes that perform particular functions. We further validated our findings on a ResNet18 model using the ImageNet dataset; here, deleting approximately 20% of the features in the average pooling layer produced targeted disruptions in specific groups of data. Additionally, our analysis confirmed that incorporating dropout in a neural network increases the redundancy, or total correlation, in the representations of later layers, an effect that was not observed in the first layer.

We believe that this approach can be used to further explore and interpret the inner workings of neural networks. For example, Local CorEx could be used to study robustness in neural networks by identifying features the network associates with a class and use them to create adversarial data points to improve model performance in a manner conceptually similar to work done by [41, 42]. Additionally, Local CorEx can be used as part of exploratory data analysis to detect variable interactions in different regions of the data

manifold such as in ecological systems [19,20], collaborations [21], the human brain [17,18], and any network-based data including biological networks. For future work, we believe this method can be further adapted as a visualization tool to aid in summarizing complex datasets.